

Half-wave plate simulation

Divyesh Chauhan (AU25L20002)

2nd March, 2026

① Introduction

② Heatmap

③ Real components

④ Complex components

- ① A half wave plate is an optical element made from a birefringent material which for a particular wavelength changes the phase difference between the x and y components of the electric field of light by π .
- ② As a result it preserves the intensity of that particular wavelength.
- ③ Due to the change in phase difference between x and y components by π , left circularly polarised waves become right circularly polarised waves and vice-versa. The linearly polarised waves are flipped about their slow fast axis.
- ④ Conventionally we choose the slow axis to take the negative sign from $e^{i\pi}$ factor.

- 1 The script from which this code is used to produce a video of an evolving heatmap of intensity.
- 2 The function `I_1` gives the intensity in the region before the half-wave plate.

```
1 def I_1 (z , v , lamd , t ) :  
2     v_x = v [0]* np.exp (1j*2*np.pi*(c*t  
        -z)/lamd)  
3     v_y = v [1]* np.exp (1j*2*np.pi*(c*t  
        -z)/lamd)  
4     return np.real (v_x)**2 + np.real (v_y)**2
```

- 3 The function `I_2` gives the intensity in the region inside the half-wave plate.

```

1 def I_2 (z,v,lamd,phi_x,phi_y,t):
2     v_x = v [0]* np.exp (1j*2*np.pi*c*t/
        lamd-phi_x*z)
3     v_y = v [1]* np.exp (1j*2*np.pi*c*t/
        lamd-phi_y*z)
4     return np.real (v_x)**2 + np.real (
        v_y)**2

```

- ④ The function I_3 gives the intensity in the region after the half-wave plate.

```

1 def I_3 (z,v,d,lamd,phi_x,phi_y,t):
2     v_x = v [0]* np.exp ( 1j*2*np.pi*(c*
        t-z+d)/lamd - phi_x*d )
3     v_y = v [1]* np.exp ( 1j*2*np.pi*(c*
        t-z+d)/lamd - phi_y*d )

```

```

4      return np.real (v_x)**2 + np.real (
          v_y)**2

```

- ⑤ Using boolean operations the function 'intensity' produces a piecewise function to calculate the square of the real amplitude.

```

1  def intensity (z,n_x,n_y,lamd,d,v,t):
2      phi_x = 1j*2*np.pi*n_x/lamd
3      phi_y = 1j*2*np.pi*n_y/lamd
4      return (z<=0)*I_1 (z,v,lamd,t) + np.
          logical_and (z>0,z<d)*I_2 (z,v,
          lamd,phi_x,phi_y,t) + (z>=d)* I_3
          (z,v,d,lamd,phi_x,phi_y,t)

```

- ⑥ The rotation angle doesn't impact the intensity so we will ignore it.

Real components I

- ① We plot the real vector on the graph and see how it evolves. From this we can see what happens to handedness, direction of polarization etc.
- ② Let λ be the wavelength such that with —

$$\lambda = \frac{d}{2 \Delta n}$$

- ③ Define $\alpha = \frac{\lambda}{\lambda'}$ where λ' is the arbitrary wavelength. Then—

$$\frac{2\pi d}{\lambda'} = \pi \alpha$$

- ④ This relationship will be utilised in this fragment of code and the next one as well.

Real components II

```
1 def update (frame):
2     v_updated = v* np.exp (1j*2*np.pi*
        frame/tot_frame)
3     c = np.cos (theta)
4     s = np.sin (theta)
5     ax.clear ()
6     ax.spines[ 'left ' ].set_position( '
        center ' )
7     ax.spines[ 'bottom' ].set_position( '
        center ' )
8     ax.spines[ 'right ' ].set_visible( False
        )
9     ax.spines[ 'top ' ].set_visible( False)
10    ax.xaxis.set_ticks_position( 'bottom '
        )
```


Real components III

```
11 ax.yaxis.set_ticks_position('left')
12 if is_after == 1:
13     plt.scatter (np.real (c*
        v_updated[0] + s*v_updated
        [1]), np.real ((-s*v_updated
        [0] + c*v_updated[1]) * np.
        exp (-1j*np.pi*alpha) ),
        color='red')
14 else :
15     plt.scatter (np.real (v_updated
        [0]), np.real (v_updated[1])
        , color='red')
16 ax.set_xlim (-1,1)
17 ax.set_ylim (-1,1)
```

Complex components I

- ① We plot the complex x and y components on the complex plane.

```
1 v_adv = v* np.exp (1j*2*np.pi*frame/  
    tot_frame)  
2 v_updated = v_adv  
3 if is_after == 1:  
4     c = np.cos (theta)  
5     s = np.sin (theta)  
6     v_updated[0] = c*v_adv[0] + s*v_adv  
        [1]  
7     v_updated[1] = -s*v_adv[0] + c*v_adv  
        [1]  
8 ax.clear ()  
9 ax.spines[ 'left' ].set_position( 'center' )
```

Complex components II

```
10 ax.spines['bottom'].set_position('center')
11 ax.spines['right'].set_visible(False)
12 ax.spines['top'].set_visible(False)
13 ax.xaxis.set_ticks_position('bottom')
14 ax.yaxis.set_ticks_position('left')
15 plt.scatter(np.real(v_updated[0]), np.
               imag(v_updated[0]), color='red',
               label='E_x')
16 plt.scatter(np.real(v_updated[1]*np.
               exp(-1j*np.pi*alpha*is_after)), np.
               imag(v_updated[1]*np.exp(-1j*np.pi*
               alpha*is_after)), color='blue',
               label='E_y')
17 plt.legend()
```

Complex components III

```
18 ax.set_xlim (-1,1)
19 ax.set_ylim (-1,1)
```